

Explicacion de cambios locales

Memoria de largo plazo para personajes y conversaciones en Luva

Generado el 28 de junio de 2026 desde /Users/cardenas/luva

Area	Cambio principal
Admin web	Nuevo panel para editar la biografia de largo plazo de un personaje.
Backend admin	Nuevos endpoints GET/POST para leer y guardar esa biografia.
Backend API	Recupera recuerdos al chatear y extrae recuerdos al terminar una conversacion.
Infraestructura	Agrega DynamoDB para biografias, env vars y permisos para Pinecone.
Scripts	Herramientas para crear/verificar el indice y vectores de Pinecone.

1. Resumen ejecutivo

Los cambios locales implementan una primera version de memoria permanente para los personajes de Luva. La memoria tiene dos fuentes: una biografia fija del personaje administrada desde el portal interno, y recuerdos por usuario/personaje extraidos de conversaciones terminadas.

La biografia se guarda como texto en DynamoDB y tambien se vectoriza en Pinecone. Los recuerdos de amistad se guardan solamente en Pinecone como hechos semanticos breves; no se guarda la transcripcion completa. Durante una conversacion, el backend recupera la biografia y los recuerdos mas relevantes para inyectarlos como contexto factual en el prompt del personaje.

Objetivo funcional

- Permitir que cada personaje tenga una biografia larga editable, separada del tagline corto del catalogo.
- Hacer que el personaje recuerde datos duraderos sobre el usuario en conversaciones futuras.
- Mantener la memoria aislada por friendshipId, con formato #, para evitar mezclar recuerdos entre usuarios.
- Hacer que fallos de memoria sean best-effort: se registran en logs, pero no bloquean el chat.

Cambios detectados en local

- 7 archivos modificados ya existentes.
- 8 archivos nuevos sin trackear relacionados con UI, admin backend, memoria y scripts.
- No hay cambios staged en el indice de Git al momento de generar este reporte.

2. Flujo funcional completo

El flujo nuevo se puede leer en tres momentos: configuracion, conversacion activa y cierre de conversacion.

Momento	Que ocurre	Resultado
Admin edita biografia	El panel llama GET/POST /v1/admin/story-characters/:id/biography	DynamoDB conserva el texto y Pinecone conserva el embedding.
Usuario conversa	advanceFriendChat consulta MemoryService para traer biografia y Top-5 Replies de los anteriores	Character Biography y Previous Replies
Chat termina	finishFriendChat manda el historial reciente a un extractor de memoria	Se almacenan hasta 8 hechos duraderos, con importancia de 1 a 5.

3. Cambios en el portal admin

admin-client.ts

Se agregan tipos y funciones cliente para hablar con los nuevos endpoints de biografia:

```
getAdminCharacterBiography(characterId)
saveAdminCharacterBiography(characterId, biography)
```

Ambas rutas usan encodeURIComponent sobre el characterId, util cuando algunos ids contienen separadores o caracteres especiales.

AdminCharacterPostsPage.tsx

La pagina de posts por personaje ahora importa CharacterBiographyPanel y lo muestra cuando existe characterId. Asi, el administrador puede gestionar posts y biografia de largo plazo desde la misma pantalla del personaje.

CharacterBiographyPanel.tsx

- Carga la biografia al montar o cuando cambia characterId.
- Mantiene estado local para biography, savedBiography, updatedAt, isLoading, isSaving, error y message.
- Deshabilita el boton Guardar cuando no hay cambios, mientras carga o mientras guarda.
- Muestra aviso de cambios sin guardar y fecha de ultima actualizacion.
- Explica dentro de la UI que esta biografia se inyecta como contexto factual y que se re-embebe en Pinecone al guardar.

4. Cambios en backend admin

backend/src/handlers/admin.ts

Se agrega una ruta nueva bajo /v1/admin/story-characters/:characterId/biography. Soporta GET y POST. Antes de operar valida que el personaje exista en CHARACTERS usando findStoryCharacter.

- GET devuelve el registro actual de biografia o una biografia vacia si no existe.
- POST parsea body.biography y llama saveCharacterBiography.
- Si falla, loguea datos utiles de diagnostico: characterId, metodo, stack parcial y variables de entorno relevantes.

backend/src/admin/biography.ts

Este modulo usa DynamoDB para persistir el texto y MemoryService para actualizar el embedding en Pinecone.

- Limita la biografia a 6000 caracteres.
- Guarda characterId, biography y updatedAt en DynamoDB.
- Si la biografia tiene texto, hace upsert del embedding en Pinecone.
- Si la biografia queda vacia, borra el vector de biografia en Pinecone.
- Si Pinecone falla despues de guardar DynamoDB, relanza el error; el texto puede quedar guardado aunque el POST responda error.

5. Cambios en backend API de chat

Extraccion de recuerdos

Se agrega extractFriendshipMemories(friend, history), que toma la conversacion reciente y pide al modelo extraer hechos duraderos sobre el learner. El prompt ignora saludos, small talk, preguntas temporales y contenido de aprendizaje de ingles.

- El resultado esperado es JSON con memories: text e importance.
- Se aceptan hasta 8 recuerdos por conversacion terminada.
- Cada texto se recorta a 500 caracteres.
- Soporta Responses API para GPT-5 o Chat Completions para otros modelos.

Recuperacion durante la conversacion

En advanceFriendChat, si hay userId, se construye friendshipId = #. Luego se consulta MemoryService para traer en paralelo la biografia y recuerdos relevantes.

- La biografia se recupera por characterId/friendId.
- Los recuerdos se buscan por similitud semantica contra el transcript y se filtran por friendshipId.
- Se recuperan topK=5 recuerdos.

- Los fallos se registran pero no impiden responder al usuario.

Uso en el prompt

generateFriendReply recibe biography y retrievedMemories. Si existen, agrega al system prompt las secciones Character Biography y Previous Memories, indicándole al modelo que las trate como contexto factual y las use naturalmente solo cuando sea relevante.

Escritura al terminar el chat

En finishFriendChat se llama extractFriendshipMemories y luego MemoryService.storeFriendshipMemories. El almacenamiento es best-effort: si falla, se loguea friends.memory.write_error y no bloquea el cierre del chat.

6. Modulo de memoria

MemoryService

backend/src/memory/memory-service.ts encapsula todo el acceso a memoria. Los handlers no dependen directamente de Pinecone ni de embeddings, lo que permite cambiar el backend vectorial mas adelante.

- character_biography: un vector deterministico por personaje, id character_biography#.
- friendship_memory: multiples vectores por friendshipId, con randomUUID para evitar colisiones.
- La metadata guarda type, ids, texto, timestamps e importance.
- La recuperacion de recuerdos filtra por type y friendshipId para aislar usuarios.

Pinecone REST wrapper

backend/src/memory/pinecone.ts implementa un cliente minimo usando fetch nativo en vez del SDK de Pinecone dentro de Lambda, para evitar problemas de bundling con esbuild.

- Lee PINECONE_API_KEY directo o PINECONE_KEY_PARAM desde SSM.
- Sanitiza comillas rectas o curvas alrededor del API key.
- Resuelve y cachea el host del indice.
- Auto-crea el indice si no existe, con dimension 1536 y metrica cosine.
- Expone solo upsert, fetch, query y deleteOne.

Embeddings y secretos

- embeddings.ts llama /v1/embeddings de OpenAI con text-embedding-3-small por defecto y dimension 1536.
- El texto a embeber se recorta a 8000 caracteres.
- secrets.ts lee parametros SecureString desde SSM con decrypt y cache en memoria por contenedor Lambda caliente.

7. Infraestructura

infra/lib/luva-stack.ts

La infraestructura agrega una tabla DynamoDB CharacterBiographiesTable con partition key characterId y removalPolicy RETAIN. Tambien prepara variables de entorno compartidas de memoria para Lambdas que la usan.

- PINECONE_KEY_PARAM apunta a /luva/pinecone/apiKey o equivalente segun ssmPath.
- PINECONE_INDEX_NAME por defecto es luva-memory.

- PINECONE_CLOUD por defecto aws y PINECONE_REGION por defecto us-east-1.
- OPENAI_EMBED_MODEL por defecto text-embedding-3-small.
- apiFn recibe env vars de memoria y permiso SSM para leer la key de Pinecone.
- adminFn recibe env vars de memoria, CHARACTER_BIOGRAPHIES_TABLE_NAME, permisos DynamoDB sobre la tabla y permiso SSM para Pinecone.

8. Scripts y dependencias

backend/package.json y package-lock.json

Se agrega @pinecone-database/pinecone 8.0.0. En runtime Lambda el código nuevo no usa el SDK directamente, pero ensure-pinecone-index.js si lo usa para provisionar el índice manualmente.

backend/scripts/ensure-pinecone-index.js

Script operativo para crear o asegurar el índice serverless de Pinecone antes del deploy o antes del primer uso real. Usa dimension 1536, métrica cosine y waitUntilReady.

backend/scripts/check-biography.js

Script de diagnóstico para verificar que exista el vector character_biography# y listar vectores de tipo character_biography. Usa directamente la API REST de Pinecone con API version 2025-01.

backend/src/memory/README.md

Documenta arquitectura, flujo de lectura/escritura, metadata guardada, setup inicial y variables de entorno requeridas.

9. Configuración necesaria antes de usarlo

- Crear o tener una API key de Pinecone.
- Guardar esa key como SecureString en SSM en la ruta esperada por ssmPath, por ejemplo /luva//pinecone/apiKey; para prod, el README indica /luva/pinecone/apiKey.
- Desplegar infra para crear CharacterBiographiesTable y cablear variables/permisos.
- Crear el índice luva-memory manualmente con ensure-pinecone-index.js o dejar que se auto-crea en el primer uso.
- Confirmar que el modelo de embeddings siga siendo de 1536 dimensiones si se cambia OPENAI_EMBED_MODEL.

10. Riesgos y observaciones técnicas

- Consistencia parcial: saveCharacterBiography escribe primero DynamoDB y luego Pinecone. Si Pinecone falla, el endpoint responde error pero DynamoDB puede haber quedado actualizado.
- Duplicados de recuerdos: storeFriendshipMemories siempre inserta nuevos vectores con randomUUID. No hay deduplicación semántica todavía.
- Costo/latencia: cada avance de chat puede hacer embeddings y query a Pinecone; cada cierre puede llamar a un LLM extractor y luego embeddings por memoria extraída.
- Privacidad: el código evita guardar la transcripción cruda, pero si guarda hechos personales sobre el usuario. Conviene revisar políticas de retención, borrado y consentimiento.
- Dependencia de configuración externa: sin SSM Pinecone key o sin permisos, la memoria falla. El chat tolera fallos, pero el guardado admin de biografía puede responder 500.

- El package `@pinecone-database/pinecone` queda como dependencia del backend por scripts, aunque el runtime Lambda usa REST directo. Conviene confirmar que no infle innecesariamente el bundle.

11. Lista de archivos tocados

- **Modificado** - `admin/src/features/admin/api/admin-client.ts`: Cliente admin para GET/POST de biografia.
- **Modificado** - `admin/src/features/admin/ui/AdminCharacterPostsPage.tsx`: Inserta el panel de biografia en la pantalla del personaje.
- **Nuevo** - `admin/src/features/admin/ui/CharacterBiographyPanel.tsx`: Editor UI de biografia de largo plazo.
- **Modificado** - `backend/package.json` y `backend/package-lock.json`: Agrega SDK de Pinecone para scripts.
- **Modificado** - `backend/src/handlers/admin.ts`: Nuevos endpoints admin de biografia.
- **Nuevo** - `backend/src/admin/biography.ts`: Persistencia DynamoDB y sincronizacion Pinecone de biografias.
- **Modificado** - `backend/src/handlers/api.ts`: Lectura/escritura de memoria en flujo de chat.
- **Nuevo** - `backend/src/memory/*`: Servicio de memoria, REST Pinecone, embeddings, SSM y README.
- **Nuevo** - `backend/scripts/ensure-pinecone-index.js`: Provisiona indice Pinecone.
- **Nuevo** - `backend/scripts/check-biography.js`: Diagnostica vectores de biografia.
- **Modificado** - `infra/lib/luva-stack.ts`: DynamoDB, env vars y permisos para memoria.

12. Conclusion

En conjunto, los cambios convierten el chat de personajes en un sistema con memoria persistente: una parte controlada por administradores y otra aprendida de interacciones reales. La implementacion esta encapsulada en `MemoryService`, usa Pinecone para busqueda semantica y DynamoDB para editar/recargar biografias desde admin.

Antes de desplegar, lo mas importante es validar configuracion de Pinecone en SSM, permisos de Lambda, dimension del indice y comportamiento ante reintentos cuando DynamoDB se actualiza pero Pinecone falla.